Neuromorphic computing practical 2

By Karan Chand

s1033357

Basic imports

```
!pip install brian2
from brian2 import *
# Note that this type of importing is very bad practice in Python,
# but a lot of brian's API relies on it...
# So we'll just have to cope with it! =)
import numpy as np
%matplotlib inline
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Collecting brian2
  Downloading Brian2-2.5.1-cp37-cp37m-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (1.5 MB)
     |████████████████████████████████| 1.5 MB 2.1 MB/s
Requirement already satisfied: sympy>=1.2 in /usr/local/lib/python3.7/dist-packages (from brian2) (1.7.1)
Requirement already satisfied: pyparsing in /usr/local/lib/python3.7/dist-packages (from brian2) (3.0.9)
Requirement already satisfied: setuptools>=24.2 in /usr/local/lib/python3.7/dist-packages (from brian2) (57.4.0)
Requirement already satisfied: cython>=0.29 in /usr/local/lib/python3.7/dist-packages (from brian2) (0.29.32)
Requirement already satisfied: jinja2>=2.7 in /usr/local/lib/python3.7/dist-packages (from brian2) (2.11.3)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.7/dist-packages (from brian2) (1.21.6)
Requirement already satisfied: MarkupSafe>=0.23 in /usr/local/lib/python3.7/dist-packages (from jinja2>=2.7->brian2) (2.0.1)
Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.7/dist-packages (from sympy>=1.2->brian2) (1.2.1)
Installing collected packages: brian2
Successfully installed brian2-2.5.1
```

Read through the basic introduction first: https://brian2.readthedocs.io/en/stable/resources/tutorials/1-intro-to-brian-neurons.html

In this notebook, we are going to use Brian to simulate different ways of representing information in networks.

Basic settings

Make sure to run the below to make the simulation timestep small enough.

If you have time you could try making this larger or smaller and see what happens to the networks (this is not graded).

```
defaultclock.dt = 0.01*ms
```

Exercise 1: Representing signals in one or several independent neurons

Single neuron example and basic explanation

```
start_scope() # this basically resets the simulation for the current cell,
              # and should be used in every notebook cell.

# Network settings
N = 1 # number of neurons
tau = 10*ms # membrane time constant
sigma = 0.1 # membrane noise level

# Network equations
# In Brian you can simply define the equation for your neural model and synapses, and the simulator
# will then take care of simulating it for you:
eqs = '''
dv/dt = -v/tau + I/tau + sigma*xi*tau**-0.5 : 1
freqs : Hz
magnitudes : 1
I = 1+magnitudes*cos(pi*2*freqs*t) : 1
'''
# Note that Brian expects you to define variables in terms of their units, and that you always have to be
# consistent with this, otherwise Brian will throw an error.

# Define neural group
G = NeuronGroup(N, eqs, threshold='v>.5', reset='v=-.55', method='euler')
G.v = 'rand()-0.5' # random initial voltages

# Set stimulus settings
G.freqs = 5*np.ones(N)*Hz
G.magnitudes = 1

## Defining a decoder
# The below defines a neuron that never spikes, but shows the net effect incoming spikes would
# have on the voltage of a downstream neuron.
# If we connect this up to the network above it turns out that is a pretty good decoder,
# since it basically filters the incoming spikes across time!
# By recording the voltage from the Gdecode neuron we can get a pretty good estimate of the input signal.
eqs_decoder = '''
dv/dt = -v/tau : 1
'''
Gdecode = NeuronGroup(1, eqs_decoder, method='euler') # a neuron without spiking
# Connect all neurons in G to the decoder
Sdecode = Synapses(G, Gdecode, on_pre='v+=1')
Sdecode.connect(i=range(N), j=0)

# A
state_monitor = StateMonitor(G, True, record=0)
spike_monitor = SpikeMonitor(G)
decoder_monitor = StateMonitor(Gdecode, True, record=0)

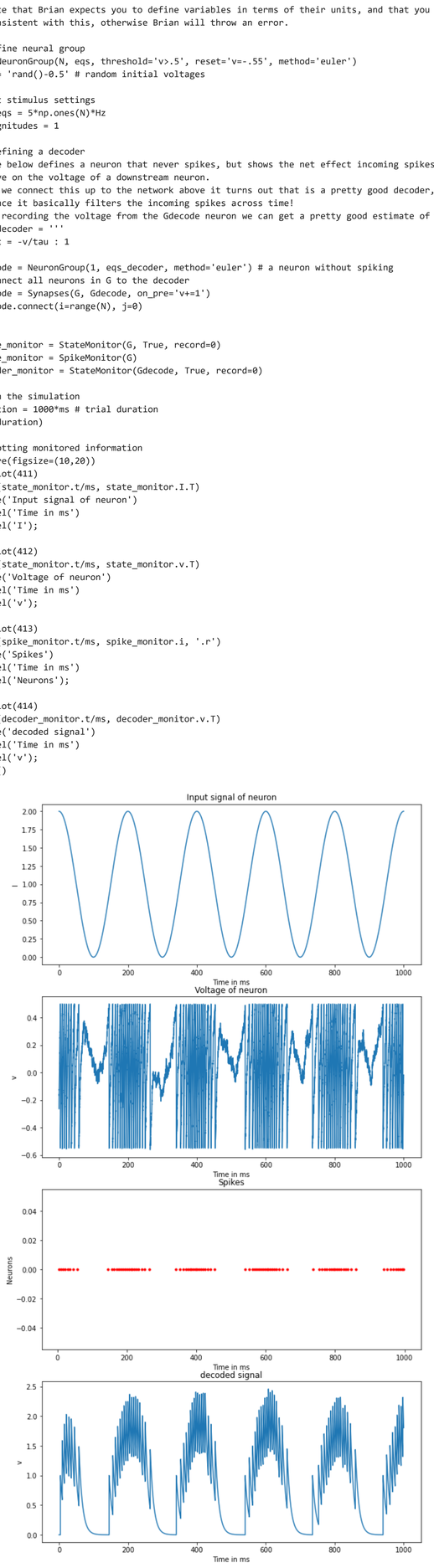
# Run the simulation
duration = 1000*ms # trial duration
run(duration)

# Plotting monitored information
figure(figsize=(10,20))
subplot(411)
plot(state_monitor.t/ms, state_monitor.I.T)
title('Input signal of neuron')
xlabel('Time in ms')
ylabel('I');

subplot(412)
plot(state_monitor.t/ms, state_monitor.v.T)
title('Voltage of neuron')
xlabel('Time in ms')
ylabel('v');

subplot(413)
plot(spike_monitor.t/ms, spike_monitor.i, '.r')
title('Spikes')
xlabel('Time in ms')
ylabel('Neurons');

subplot(414)
plot(decoder_monitor.t/ms, decoder_monitor.v.T)
title('decoded signal')
xlabel('Time in ms')
ylabel('v');
show()
```




```
start_scope() # this basically resets the simulation for the current cell,
              # and should be used in every notebook cell.

# Network settings
N = 10 # number of neurons
tau = 10*ms # membrane time constant
sigma = 0.5 # membrane noise level

# Network equations
# In Brian you can simply define the equation for your neural model and synapses, and the simulator
# will then take care of simulating it for you:
eqs = '''
dv/dt = -v/tau + I/tau + sigma*xi*tau**-0.5 : 1
freqs : Hz
magnitudes : 1
I = 1+magnitudes*cos(pi*2*freqs*t) : 1
'''
# Note that Brian expects you to define variables in terms of their units, and that you always have to be
# consistent with this, otherwise Brian will throw an error.

# Define neural group
G = NeuronGroup(N, eqs, threshold='v>.5', reset='v=-.55', method='euler')
G.v = 'rand()-0.5' # random initial voltages

# Set stimulus settings
G.freqs = 5*np.ones(N)*Hz
G.magnitudes = 1

## Defining a decoder
# The below defines a neuron that never spikes, but shows the net effect incoming spikes would
# have on the voltage of a downstream neuron.
# If we connect this up to the network above it turns out that is a pretty good decoder,
# since it basically filters the incoming spikes across time!
# By recording the voltage from the Gdecode neuron we can get a pretty good estimate of the input signal.
eqs_decoder = '''
dv/dt = -v/tau : 1
```

```
'''
Gdecode = NeuronGroup(1, eqs_decoder, method='euler') # a neuron without spiking
# Connect all neurons in G to the decoder
Sdecode = Synapses(G, Gdecode, on_pre='v+=1')
Sdecode.connect(i=range(N), j=0)

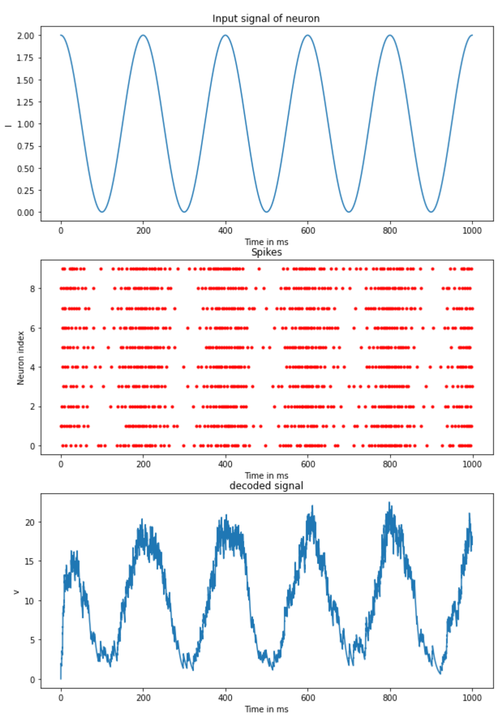
state_monitor = StateMonitor(G, True, record=0)
spike_monitor = SpikeMonitor(G)
decoder_monitor = StateMonitor(Gdecode, True, record=0)

# Run the simulation
duration = 1000*ms # trial duration
run(duration)

# Plotting monitored information
figure(figsize=(10,15))
subplot(311)
plot(state_monitor.t/ms, state_monitor.I.T)
title('Input signal of neuron')
xlabel('Time in ms')
ylabel('I');

subplot(312)
plot(spike_monitor.t/ms, spike_monitor.i, '.r')
title('Spikes')
xlabel('Time in ms')
ylabel('Neuron index');

subplot(313)
plot(decoder_monitor.t/ms, decoder_monitor.v.T)
title('decoded signal')
xlabel('Time in ms')
ylabel('v');
show()
```



B.

The decoded signal has its peaks at around 20 now, which is around 10 times higher than 1 neuron. This is because spikes are added and around 10 times more neurons are firing. The decoded signal could track the signal better by averaging across the signal, maybe by having bins. The signal amplitude is way higher than the input signal, so maybe this will help. More neurons do seem to track the general flow of the signal better, as there is less noise.

C

The spiking patterns resemble the biological network in the slides, as the decoded signal still is distinguishable with high noise (0.5).

Exercise 2: Representing signals in randomly connected neurons

```
def visualise_connectivity(S):
    Ns = len(S.source)
    Nt = len(S.target)
    figure(figsize=(12, 4))
    subplot(121)
    plot(zeros(Ns), arange(Ns), 'ok', ms=10)
    plot(ones(Nt), arange(Nt), 'ok', ms=10)
    for i, j in zip(S.i, S.j):
        plot([0, 1], [i, j], '-k')
    xticks([0, 1], ['Source', 'Target'])
    ylabel('Neuron index')
    xlim(-0.1, 1.1)
    ylim(-1, max(Ns, Nt))
    subplot(122)
    plot(S.i, S.j, 'ok')
    xlim(-1, Ns)
    ylim(-1, Nt)
    xlabel('Source neuron index')
    ylabel('Target neuron index')


## Adding connectivity
# to add synapses you can use the synapse object
# (https://brian2.readthedocs.io/en/stable/resources/tutorials/2-intro-to-brian-synapses.html):
# S = Synapses(A, B, 'w : 1', on_pre='v+=w')
#
# This creates a synapses object between neural groups A and B.
# With each spike from A to B through a synapse the voltage goes up by 'w'.
# However, we also have to connect up specific neurons in both groups:
# S.connect('i!=j',  p=1) # this gives each possible connection (except self connections) a probability of 1 of existing
#
# You could also come up with other connection rules of course!
#
# Finally we need to define what the strength of the weights is. This could be a fixed number:
# S.w = '1'
#
# Or a random number:
# S.w = 'rand()'
#
# Where rand() acts like np.random.rand() and generates a random number between 0 and 1 for each connection.
# Other equations are also possible of course, see the documentation.
#
# For an example in this notebook, see how we connected the Gdecode NeuronGroup to the G NeuronGroup above.
# For more in-depth explanations and examples, see the Brian documentation.

start_scope() # this basically resets the simulation for the current cell,
              # and should be used in every notebook cell.

# Network settings
N = 10 # number of neurons
tau = 10*ms # membrane time constant
sigma = 0.1 # membrane noise level

# Network equations
# In Brian you can simply define the equation for your neural model and synapses, and the simulator
# will then take care of simulating it for you:
eqs = '''
dv/dt = -v/tau + I/tau + sigma*xi*tau**-0.5 : 1
freqs : Hz
magnitudes : 1
I = 1+magnitudes*cos(pi*2*freqs*t) : 1
'''
# Note that Brian expects you to define variables in terms of their units, and that you always have to be
# consistent with this, otherwise Brian will throw an error.

# Define neural group
G = NeuronGroup(N, eqs, threshold='v>.5', reset='v=-.55', method='euler')
G.v = 'rand()-0.5' # random initial voltages
S = Synapses(G, G, 'w : 1', on_pre='v+=w')
S.connect(condition='i!=j', p=0.25)
S.w = 'rand()'

# Set stimulus settings
G.freqs = 5*np.ones(N)*Hz
G.magnitudes = 1

## Defining a decoder
# The below defines a neuron that never spikes, but shows the net effect incoming spikes would
# have on the voltage of a downstream neuron.
# If we connect this up to the network above it turns out that is a pretty good decoder,
# since it basically filters the incoming spikes across time!
# By recording the voltage from the Gdecode neuron we can get a pretty good estimate of the input signal.
eqs_decoder = '''
dv/dt = -v/tau : 1
'''
Gdecode = NeuronGroup(1, eqs_decoder, method='euler') # a neuron without spiking
# Connect all neurons in G to the decoder
Sdecode = Synapses(G, Gdecode, on_pre='v+=1')
Sdecode.connect(i=range(N), j=0)

state_monitor = StateMonitor(G, True, record=0)
spike_monitor = SpikeMonitor(G)
decoder_monitor = StateMonitor(Gdecode, True, record=0)

# Run the simulation
duration = 1000*ms # trial duration
run(duration)

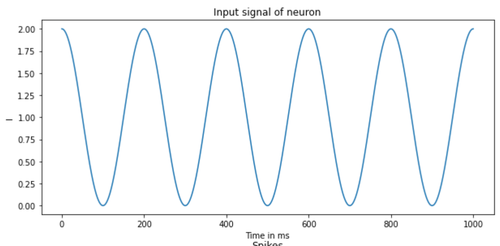
# Plotting monitored information
figure(figsize=(10,15))

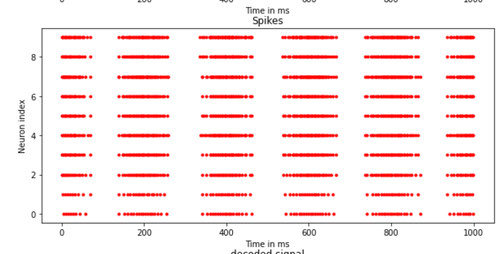
subplot(311)
plot(state_monitor.t/ms, state_monitor.I.T)
title('Input signal of neuron')
xlabel('Time in ms')
ylabel('I');

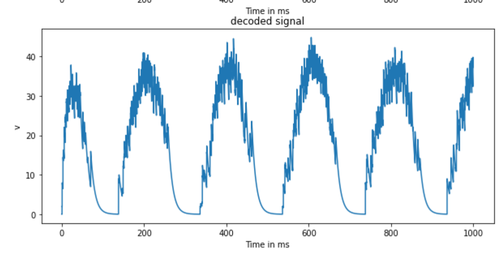
subplot(312)
plot(spike_monitor.t/ms, spike_monitor.i, '.r')
title('Spikes')
xlabel('Time in ms')
ylabel('Neuron index');

subplot(313)
plot(decoder_monitor.t/ms, decoder_monitor.v.T)
title('decoded signal')
xlabel('Time in ms')
ylabel('v');
show()

visualise_connectivity(S)
```









C

The network activity is higher, as some neurons are connected. This means that neuros will fire sooner than before, as thresholds are sooner reached. Thresholds are sooner reached because of the positive weights as well, as they are higher than before. High spike firing leads to the decoded signal being even higher than before (around 40).

Exercise 3: Representing signals in balanced networks

```
start_scope() # this basically resets the simulation for the current cell,
              # and should be used in every notebook cell.

# Network settings
N = 10 # number of neurons
tau = 10*ms # membrane time constant
sigma = 0.1 # membrane noise level

# Network equations
# In Brian you can simply define the equation for your neural model and synapses, and the simulator
# will then take care of simulating it for you:
eqs = '''
dv/dt = -v/tau + I/tau + sigma*xi*tau**-0.5 : 1
freqs : Hz
magnitudes : 1
I = 1+magnitudes*cos(pi*2*freqs*t) : 1
'''
# Note that Brian expects you to define variables in terms of their units, and that you always have to be
# consistent with this, otherwise Brian will throw an error.

# Define neural group
G = NeuronGroup(N, eqs, threshold='v>.5', reset='v=-.55', method='euler')
G.v = 'rand()-0.5' # random initial voltages
S = Synapses(G, G, 'w : 1', on_pre='v+=w')
S.connect(condition='i!=j', p=0.25)
S.w = 'rand()-0.5'

# Set stimulus settings
G.freqs = 5*np.ones(N)*Hz
G.magnitudes = 1

## Defining a decoder
# The below defines a neuron that never spikes, but shows the net effect incoming spikes would
# have on the voltage of a downstream neuron.
# If we connect this up to the network above it turns out that is a pretty good decoder,
# since it basically filters the incoming spikes across time!
# By recording the voltage from the Gdecode neuron we can get a pretty good estimate of the input signal.
eqs_decoder = '''
dv/dt = -v/tau : 1
'''
Gdecode = NeuronGroup(1, eqs_decoder, method='euler') # a neuron without spiking
# Connect all neurons in G to the decoder
Sdecode = Synapses(G, Gdecode, on_pre='v+=1')
Sdecode.connect(i=range(N), j=0)

state_monitor = StateMonitor(G, True, record=0)
spike_monitor = SpikeMonitor(G)
decoder_monitor = StateMonitor(Gdecode, True, record=0)

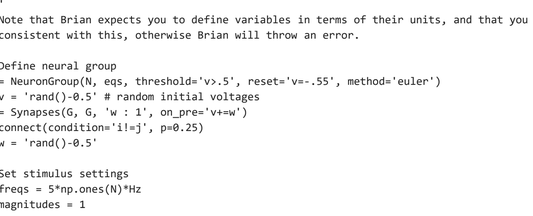
# Run the simulation
duration = 1000*ms # trial duration
run(duration)

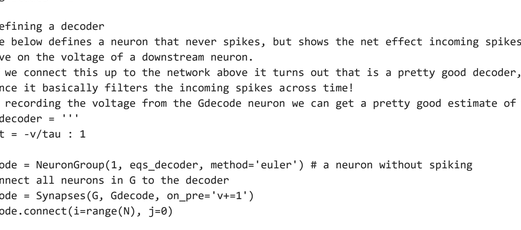
# Plotting monitored information
figure(figsize=(10,15))
subplot(311)
plot(state_monitor.t/ms, state_monitor.I.T)
title('Input signal of neuron')
xlabel('Time in ms')
ylabel('I');

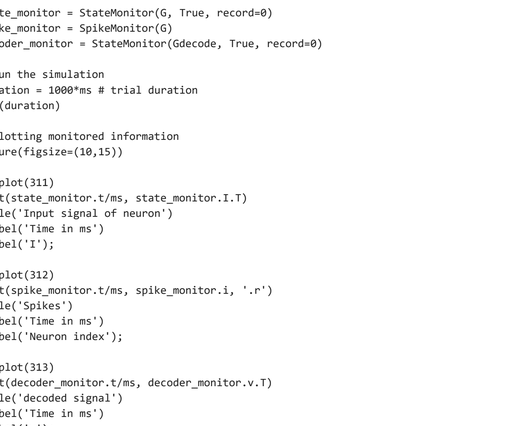
subplot(312)
plot(spike_monitor.t/ms, spike_monitor.i, '.r')
title('Spikes')
xlabel('Time in ms')
ylabel('Neuron index');

subplot(313)
plot(decoder_monitor.t/ms, decoder_monitor.v.T)
title('decoded signal')
xlabel('Time in ms')
ylabel('v');
show()

visualise_connectivity(S)
```









C

Spiking occurs less, which leads to a decrease of amplitude in the decoded signal. This is because the weights are lower compared to the previous question and some are even negative, which leads to inhibitive neurons and less firing. The decoded signal does represent the input signal well, eventhough it is still noisy.

Exercise 4: Coordinated spiking

```
start_scope() # this basically resets the simulation for the current cell,
              # and should be used in every notebook cell.

# Network settings
N = 10 # number of neurons
tau = 10*ms # membrane time constant
sigma = 0.1 # membrane noise level

# Network equations
# In Brian you can simply define the equation for your neural model and synapses, and the simulator
# will then take care of simulating it for you:
eqs = '''
dv/dt = -v/tau + I/tau + sigma*xi*tau**-0.5 : 1
freqs : Hz
magnitudes : 1
I = 1+magnitudes*cos(pi*2*freqs*t) : 1
'''
# Note that Brian expects you to define variables in terms of their units, and that you always have to be
# consistent with this, otherwise Brian will throw an error.

# Define neural group
G = NeuronGroup(N, eqs, threshold='v>.5', reset='v=-.55', method='euler')
G.v = 'rand()-0.5' # random initial voltages
```

```
S = Synapses(G, G, 'w : 1', on_pre='v+=w')
S.connect(condition='i!=j', p=0.25)
S.w = 'rand()-1'

# Set stimulus settings
G.freqs = 5*np.ones(N)*Hz
G.magnitudes = 1

## Defining a decoder
# The below defines a neuron that never spikes, but shows the net effect incoming spikes would
# have on the voltage of a downstream neuron.
# If we connect this up to the network above it turns out that is a pretty good decoder,
# since it basically filters the incoming spikes across time!
# By recording the voltage from the Gdecode neuron we can get a pretty good estimate of the input signal.
eqs_decoder = '''
dv/dt = -v/tau : 1
'''
Gdecode = NeuronGroup(1, eqs_decoder, method='euler') # a neuron without spiking
# Connect all neurons in G to the decoder
Sdecode = Synapses(G, Gdecode, on_pre='v+=1')
Sdecode.connect(i=range(N), j=0)

state_monitor·=·StateMonitor(G,·True,·record=0)
spike_monitor·=·SpikeMonitor(G)
decoder_monitor·=·StateMonitor(Gdecode,·True,·record=0)

#·Run·the·simulation
duration·=·1000*ms·#·trial·duration
run(duration)

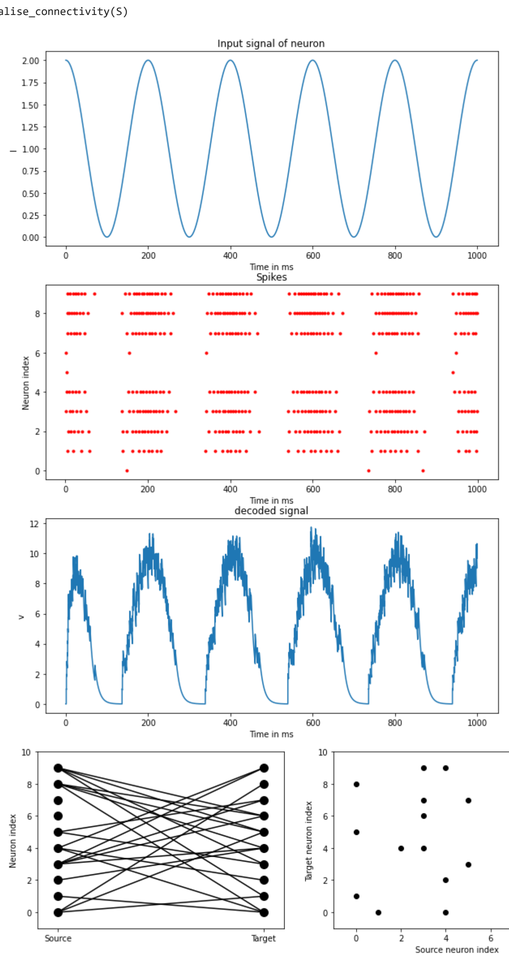
#·Plotting·monitored·information
figure(figsize=(10,15))

subplot(311)
plot(state_monitor.t/ms,·state_monitor.I.T)
title('Input·signal·of·neuron')
xlabel('Time·in·ms')
ylabel('I');

subplot(312)
plot(spike_monitor.t/ms,·spike_monitor.i,·'.r')
title('Spikes')
xlabel('Time·in·ms')
ylabel('Neuron·index');

subplot(313)
plot(decoder_monitor.t/ms,·decoder_monitor.v.T)
title('decoded·signal')
xlabel('Time·in·ms')
ylabel('v');
show()

visualise_connectivity(S)
```



C

The spiking pattern is very sparse compared to the previous examples. Some neurons only fire a few times. The decoded signal is still representative of the input signal, but it is back to ~10 times higher amplitude. This is because the neurons have a negative weight and inhibit eachother due to connectivity. This makes the spiking pattern very different from before, but the output signal not that much as thresholds are still reached.

```
start_scope() # this basically resets the simulation for the current cell,
              # and should be used in every notebook cell.

# Network settings
N = 10 # number of neurons
tau = 10*ms # membrane time constant
sigma = 0.1 # membrane noise level

# Network equations
# In Brian you can simply define the equation for your neural model and synapses, and the simulator
# will then take care of simulating it for you:
eqs = '''
dv/dt = -v/tau + I/tau + sigma*xi*tau**-0.5 : 1
freqs : Hz
magnitudes : 1
I = 1+magnitudes*cos(pi*2*freqs*t) : 1
'''
# Note that Brian expects you to define variables in terms of their units, and that you always have to be
# consistent with this, otherwise Brian will throw an error.

# Define neural group
G = NeuronGroup(N, eqs, threshold='v>.5', reset='v=-.55', method='euler')
G.v = 'rand()-0.5' # random initial voltages
S = Synapses(G, G, 'w : 1', on_pre='v+=w')
S.connect(condition='i!=j', p=0.3)
S.w = '-1'

# Set stimulus settings
G.freqs = 5*np.ones(N)*Hz
G.magnitudes = 1

## Defining a decoder
# The below defines a neuron that never spikes, but shows the net effect incoming spikes would
# have on the voltage of a downstream neuron.
# If we connect this up to the network above it turns out that is a pretty good decoder,
# since it basically filters the incoming spikes across time!
# By recording the voltage from the Gdecode neuron we can get a pretty good estimate of the input signal.
eqs_decoder = '''
dv/dt = -v/tau : 1
'''
Gdecode = NeuronGroup(1, eqs_decoder, method='euler') # a neuron without spiking
# Connect all neurons in G to the decoder
Sdecode = Synapses(G, Gdecode, on_pre='v+=1')
Sdecode.connect(i=range(N), j=0)

state_monitor = StateMonitor(G, True, record=0)
spike_monitor = SpikeMonitor(G)
decoder_monitor = StateMonitor(Gdecode, True, record=0)

# Run the simulation
duration = 1000*ms # trial duration
run(duration)

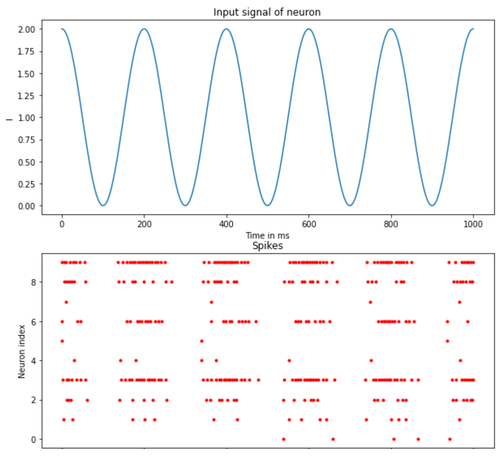
# Plotting monitored information
figure(figsize=(10,15))

subplot(311)
plot(state_monitor.t/ms, state_monitor.I.T)
title('Input signal of neuron')
xlabel('Time in ms')
ylabel('I');

subplot(312)
plot(spike_monitor.t/ms, spike_monitor.i, '.r')
title('Spikes')
xlabel('Time in ms')
ylabel('Neuron index');

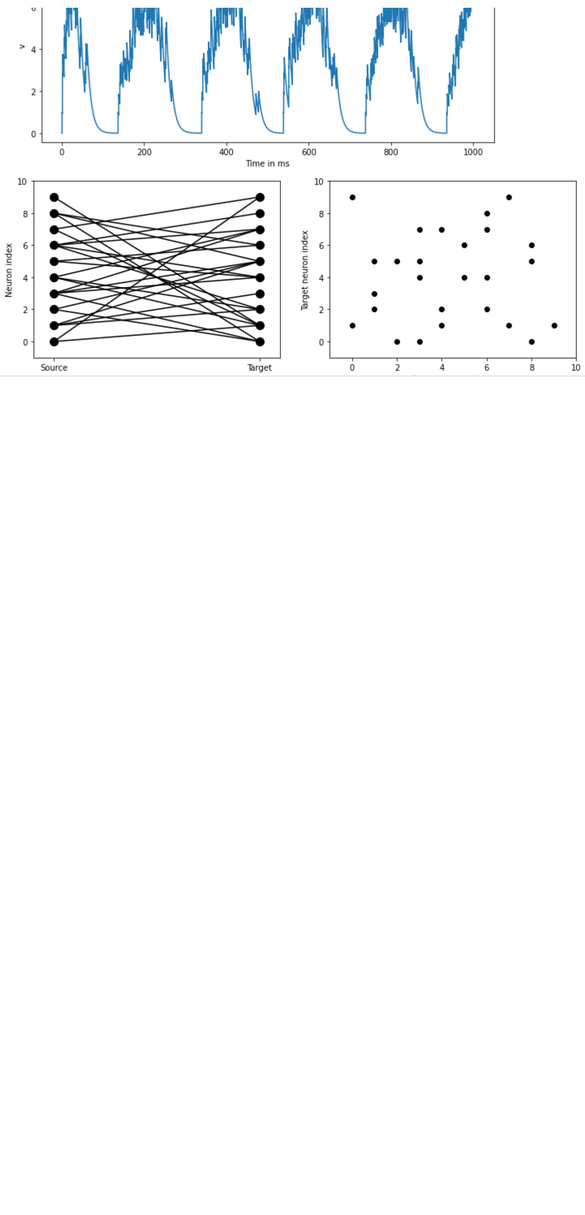
subplot(313)
plot(decoder_monitor.t/ms, decoder_monitor.v.T)
title('decoded signal')
xlabel('Time in ms')
ylabel('v');
show()

visualise_connectivity(S)
```



F

Thresholds are reached less, which leads to a sparser spiking pattern. Even less spikes occur, as the threshold is relatively high compared to the previous examples. Less firing consitutes to the 8 times larger amplitude than the input signal. The decoded signal is recognizable, but it is very noisy due to the low resolution on the x axis.



✓ 12s completed at 11:20 AM